Red Hat Summit

Browse AI-related sessions at Red Hat® Summit.

See options →

Improve GPU utilization with Kueue in OpenShift AI

How IBM achieved 90% GPU allocation in Vela

May 22, 2025 | Akram Ben Aissi, Olivier Tardieu, David Grove

Related topics: Artificial intelligence, Data science, Kubernetes

Related products: Red Hat AI, Red Hat OpenShift AI, Red Hat OpenShift

In 2024, the world ran on GPUs. Whether it was training billion-parameter foundation models, fine-tuning vision transformers, or deploying lightning-fast inference pipelines, the demand for compute—especially GPU compute—skyrocketed to levels never seen before. But while every lab, startup, and enterprise raced to get their hands on H100s and A100s, a quieter, more persistent problem kept creeping in: **GPU underutilization**.

It turns out that even with a rack full of cutting-edge silicon, you might still find yourself staring at idle GPUs.

And that's a *very* expensive view.

The GPU efficiency challenge

The H100 GPU —a favorite among machine learning (ML) practitioners and one of the most popular pieces of hardware in the world—sells for up to $40,000 on the secondary market. The price has doubled or even tripled in some regions compared to its official MSRP, driven by a supply-demand mismatch that's been dragging on for years. Designing and manufacturing these chips is an incredibly capital-intensive process, requiring bleeding-edge fabs, advanced packaging, and years of R&D. Only a handful of companies in the world can actually pull it off, and the entire market is dominated by just one or two players.

This scarcity means that every GPU minute matters. Yet in real-world ML platforms, utilization rates often hover around 30–50%—a staggeringly low figure when you consider what these chips cost. Fragmented workloads, inefficient scheduling, job queuing delays, or even simple overprovisioning all contribute to this problem. You might be paying for the Ferrari of compute, but only using it to drive to the corner store.

The issue and the frustration are even more acute when having to manage an entire fleet of racing cars and maximize their usage.

This raises the question: **What if we could drive GPU utilization above 90%?** That's not a theoretical target—it's exactly what IBM achieved on its internal ML platform, Vela, by rethinking how GPU workloads are orchestrated with OpenShift and Kueue and additional tools made by Open Source Sauce.

Let's dive in.

Meet Vela: IBM Research's ML Supercomputer

When most people think of a supercomputer, they imagine rows of bare-metal servers locked away in an ultra-cooled, badge-access data center. But IBM took a very different route with **Vela**, its first AI-optimized, cloud-native supercomputer. Since May 2022, Vela has been the backbone of IBM Research's most ambitious AI efforts—powering the training of foundation models, multi-modal systems, and other large-scale workloads that require serious compute resources.

The twist? Vela isn't a traditional supercomputer. It was designed from the ground up to be cloud-native—a virtualized system that delivers near bare-metal performance with less than 5% overhead. This allows it to combine the raw power of specialized AI hardware with the flexibility and elasticity of modern cloud infrastructure.

Each Gen-1 Vela node consists in an amazing box of:

- **Eight A100 GPUs** each with 80GB of memory, tightly coupled via NVLink and NVSwitch
- **1.5TB of RAM** and **12.8TB of local NVMe storage**
- Dual **Intel Xeon Scalable CPUs**, tuned to handle the orchestration of massive distributed training jobs

Rather than relying on exotic interconnects like InfiniBand, Vela uses standard Ethernet-based networking, arranged in a two-tier Clos topology with no oversubscription. Thanks to clever tuning of PyTorch communication patterns, network latency becomes almost a non-issue, even for models with tens of billions of parameters.

But perhaps the most radical thing about Vela is not what it's made of—it's *how* it's built and operated. Its virtualized design means IBM can easily allocate capacity between HPC and cloud-native stacks, provision infrastructure as code, and even deploy Vela-like systems across different IBM Cloud regions globally.

The result? A system that can hit **up to 90% GPU utilization** for real-world, distributed model training tasks. That's not just efficient—it's game-changing. Especially in a world where idle GPUs are measured in lost dollars per minute.

The bottlenecks: Why even Vela had idle GPUs

Quotas: Large GPU pools like the Vela Supercomputer are shared by users across continents and time zones. It is virtually impossible to coordinate GPU allocation without enforcement mechanisms. The ResourceQuotas built into Kubernetes can enforce GPU allocation limits but are inflexible. Unused quotas result in idle GPUs. Excess workloads are rejected rather than

queued. Cluster admins can enable either quotas or priorities but combining the two is a recipe for disaster.

Faults: Hardware issues like unhealthy GPUs or flaky network links are inevitable at scale. When they happen, workloads may experience failures or partial failures. While Kubernetes offers mechanisms to handle individual pod failures, these do not extend to automatically recovering distributed workloads. For example, a workload experiencing a single-node failure may stall but continue to reserve GPU capacity until explicitly evicted by the workload owner or a cluster admin.

Red Hat OpenShift AI: A clear choice

IBM built Vela on Red Hat OpenShift AI to provide a consistent, enterprise-ready platform for managing AI workloads at scale. OpenShift AI offers a well integrated distribution of machine learning tools, combining the flexibility of Kubernetes with a proven set of components tailored for AI development and operations:

- Model development tooling based on the popular **Jupyter** notebook solution
- Data pipeline orchestration frameworks
- Comprehensive serving solution supporting both single and multi-model with **KServe/vLLM**
- Monitoring and observability stack based on **Prometheus**
- Kubernetes-native workload management and GPU scheduling with **kueue**

OpenShift AI, as a packaged environment, assembled in an MLOps Suite, allows machine learning engineers and DevOps teams to collaborate using a shared platform. The tools for model development, model serving, data processing, monitoring and GPU scheduling are included by default, accelerating experimentation while reducing integration time.

And to add icing on the cake, the crucial differentiator of OpenShift AI is its fine-grained configurability and extensibility: Even though several

components are bundled together in a powerful ML distribution, it is still possible to disable them one by one to connect a specific version or build in order to take advantage of as yet unreleased features. In a sentence: tune the platform to get the most out of it by experimenting.

Enter Kueue: Smarter batch workload orchestration for Kubernetes

In Kubernetes, scheduling works great—until it doesn't.

Kubernetes was designed with general-purpose compute in mind. It's excellent at bin-packing small pods onto big nodes and can scale stateless services in a blink of an eye. But when you throw in jobs that need multiple GPUs, terabytes of memory, and might sit idle while waiting for dependencies or shared storage mounts, things can get odd very quickly.

That's where Kueue comes in.

Kueue is a Kubernetes-native job queueing system, purpose-built for ML and batch workloads. It doesn't try to reinvent the Kubernetes scheduler. Instead, it wraps around it, adding a queueing layer that gives control over when and how jobs are admitted to the cluster. Think of it as a velvet rope in front of the club—the cluster stays packed, but never overcrowded. The VIPs get priority but everybody has a turn.

Kueue works by orchestrating multiple job queues across namespaces and teams, managing quotas and priorities. It supports gang-scheduling for distributed workloads, cohort-based quota borrowing, and automatic workload suspension, which are crucial when you are trying to keep expensive GPUs humming 24/7. See Figure 1.

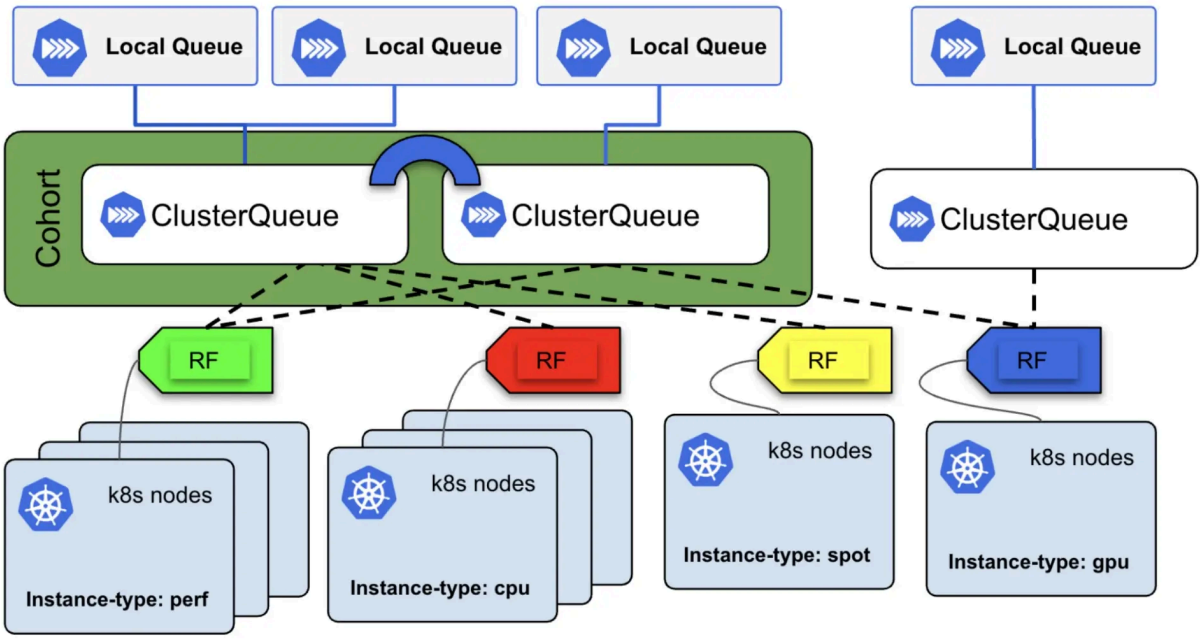

Figure 1: Illustration diagram explaining how Kueue works.

Why did IBM pick Kueue for Vela?

Because Vela's challenge is not about getting more GPUs, but rather **getting more out of the GPUs they already have**. Kueue offers a solution that is Kubernetes-native, composable, and most importantly, designed to respect the realities of modern ML workflows.

Cohorts and quota borrowing: Structured flexibility for GPU allocation

Kueue introduces a cohort-based model for resource sharing that supports both **organizational isolation** and **elastic utilization**. In this model, multiple queues can be grouped into a **cohort** when they require access to the same set of resources. Each queue receives a nominal quota, which serves as a **resource access guarantee**.

Within a cohort, workloads can **borrow unused quota** from one another to accommodate bursty demand. This mechanism addresses a common inefficiency in shared clusters: resources reserved for one namespace but underutilized cannot benefit another namespace resulting in a wasteful combination of pending jobs and idle resources. In contrast, quota borrowing allows Kueue to make opportunistic use of idle capacity without ever

violating resource access guarantees by promptly returning capacity to the quota owner upon demand.

Consider the following plot (Figure 2) generated from one Vela instance intended to run workloads in the 1-1K GPU range. The leftmost chart shows the nominal partition of the cluster among teams. The dark-gray team is allocated 22% of the GPUs. The light-gray sector (10%) corresponds to reserve capacity that is not statically assigned, but can be borrowed by any team. The middle chart shows the actual GPU allocation with less than 1% idle GPUs when the snapshot was taken. While a number of teams are not present on the cluster, other teams including dark-gray are bursting above their quotas. The rightmost chart demonstrates that without borrowing about 30% of GPUs would remain idle. Obviously, without the ability to borrow, no GPU would have been left statically unassigned. Even though, at least 20% of GPUs would still be idle.

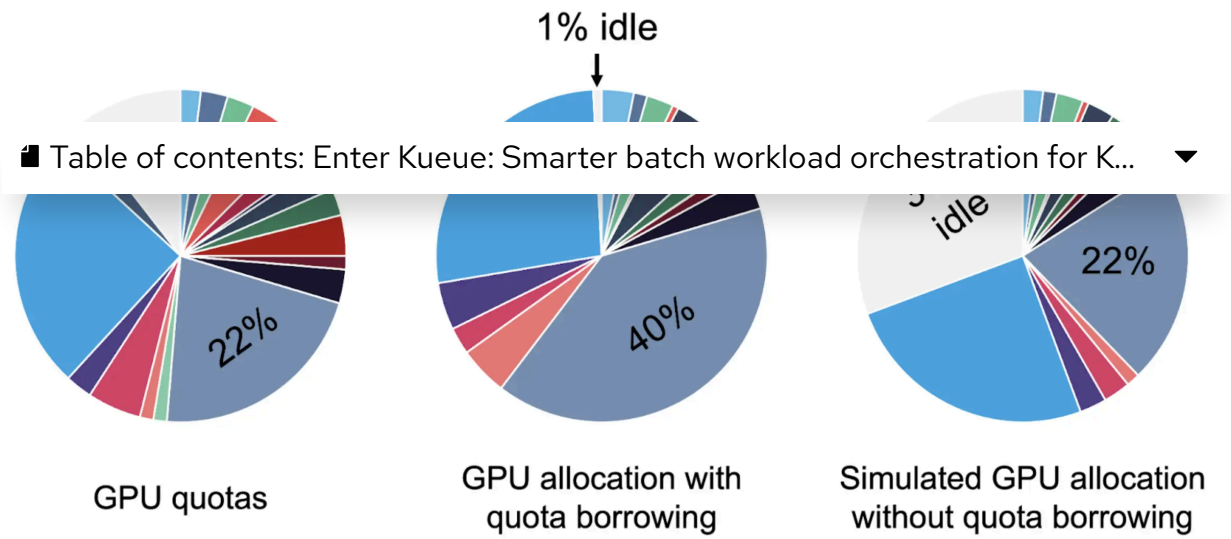

Figure 2: GPU utilization when enabling quotas and quotas borrowing.

Priority classes: Deterministic scheduling under contention

Kueue accounts for Kubernetes **priority classes**, which provide a mechanism for distinguishing between workloads based on operational importance. When resources are constrained, jobs with higher priority are scheduled first. Kueue extends the Kubernetes-native mechanisms with two essential capabilities. First, Kueue's job priorities and team quotas work together: a

high-priority job will only take precedence over lower priority jobs within its quota, i.e., within the team. Second, a preempted lower-priority job is not summarily terminated but instead **suspended** and **requeued** to be **resumed** later.

A prerequisite for this strategy is that jobs implement **checkpointing**, allowing them to resume without loss of progress rather than restart when interrupted. In practice, this requires zero effort as typical ML workloads already implement checkpointing for fault-tolerance purposes.

Before deploying Kueue, high-priority workloads on IBM Research's very first Vela cluster were handled via explicit coordination among users. Concretely, users would be asked to terminate jobs to make room for the critical workloads of the day and wait for instructions before restarting. This was not only a frustrating experience but also resulted in idle GPUs due to the accumulation of delays.

In short, by combining **queues**, **quotas**, **priorities**, **gang scheduling**, and **suspend/resume** mechanisms, Kueue provides a framework for ensuring optimal GPU allocation without sacrificing automation while guaranteeing fair resource allocation across teams. For organizations operating large-scale AI infrastructure, this set of features should be considered foundational rather than optional.

How AppWrapper, Autopilot, and MLBatch changed the game

The ambition of the Vela platform is to make it possible for workloads to take full advantage of the available GPU capacity but this by no means guarantees productive GPU utilization. GPU utilization is not easily defined since GPUs are multidimensional devices. Workloads might bottleneck on compute, memory, memory bandwidth, host-to-device or device-to-device communication, can throttle due to thermal safeguards, etc. To achieve 90% GPU utilization, we need to combine GPU allocation virtually at a constant 100% level with 90% workload efficiency. While the platform cannot magically fix algorithmic inefficiencies, it can and therefore must address

inefficiencies due to infrastructure deficiencies such as hardware failures or degraded performance.

And the magic can start to operate with the mixture of the following tools: **MLBatch**, an open-source project originating from IBM Research and deployed on Vela that complements **Kueue** with fault detection and recovery capabilities respectively provided by **Autopilot** and **AppWrapper**.

Let's see how they all work together.

Infrastructure monitoring with autopilot

Autopilot continuously monitors and evaluates GPUs, network, and storage health on the cluster, periodically running lightweight background checks. It can also run deep diagnostics on GPUs when idle. Detected issues are automatically assigned into one of two buckets:

- **Faults** that require immediate workload suspension,
- **Anomalies** that should be investigated but do not warrant workload suspension at this stage.

For example, an ECC error on a GPU is considered a fault, whereas an excursion above the thermal threshold is an anomaly.

In all cases, Autopilot flags faulty devices alerting cluster admins and steering incoming workloads away from them.

While native Kubernetes fault management mechanisms operate at the node level, Autopilot is finer-grained making it possible for instance to continue to leverage seven healthy GPUs on a node where a single GPU has failed.

Robust job life cycle management with AppWrapper

While Kueue addresses the scheduling of distributed jobs and Kubernetes the failure of individual pods through retry policies, the combination does not guarantee the operational integrity of multi-pod jobs—particularly in the presence of partial failures. This responsibility is filled by AppWrapper.

Prior to deploying AppWrapper to Vela clusters, we routinely observed high GPU allocation coinciding with low utilization due to the following issues:

- An infrastructure failure such as a GPU fatal error may cause a job to get stuck instead of failing outright.
- Deleting a pod in order to replace it may fail due long grace periods or Kubernetes control plane issues.
- Replacing a single or a few failed pods in a large job may not be enough to repair the job.

AppWrapper addresses these concerns as follows:

- **AppWrapper decides that a job has failed** by monitoring both the infrastructure diagnostics provided by Autopilot and the pod statuses reported by Kubernetes.
- **AppWrapper fully recycles the job** by *forcefully* terminating and recreating all the job components, in particular all its pods.

The diagram in Figure 3 depicts the lifecycle of an AppWrapper. The outer loop consisting of the Suspended, Admitted, and Suspending states is managed by Kueue, while the inner loop consisting of the Resuming, Running, and Resetting states is managed by the AppWrapper controller. In particular, the AppWrapper controller handles workload retries without releasing and reacquiring Kueue quotas, hence without moving retried workloads to the back of the cluster queue.

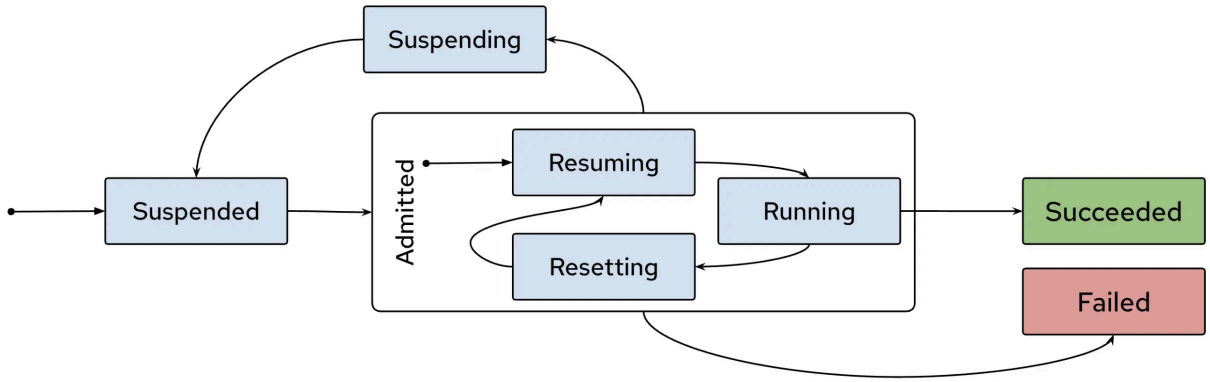

Figure 3: AppWrapper life cycle.

The results: Hitting 90% GPU utilization

By combining Kueue's resource management with the fault detection and recovery capabilities of Autopilot and AppWrapper, MLBatch creates a closed-loop control system for AI workload execution. Faults are no longer tolerated silently. Resources are promptly reclaimed and reassigned. Scheduling decisions adapt to the state of the system in near real-time.

In aggregate, these tools transform Vela into a high-efficiency AI platform capable of sustaining **90% GPU utilization** under active load. The combination of team and fault management proved essential to overcoming the limitation of core Kubernetes when applied to large-scale AI infrastructure.

Conclusion: Efficiency as a competitive advantage

As AI workloads scale, maximizing GPU utilization has become a critical differentiator. IBM's experience with Vela shows that even well-designed infrastructure can face underutilization without effective scheduling, failure recovery, and orchestration.

The combination of Kueue and MLBatch enables the Vela platform to serve many more teams and workloads—not through added hardware—but through smarter resource allocation and fault management. And all of these components are open source (Apache 2.0 licensed), which makes them easily available for you to leverage to create Vela-like systems for your organization. We hope you find them useful and welcome your feedback and contributions to make the stack even better!

In a context where GPUs remain scarce and demand continues to grow, operational efficiency is no longer optional—it is a core advantage.

References

Project GitHubs:

- Kueue

- MLBatch
- AppWrapper
- Autopilot

Related talks:

- MLBatch Tutorial at KubeCon EU 2025
- Autopilot talk at KubeCon EU 2025

Related Posts

What is GPU programming?

GPU enablement on MicroShift

How to use AMD GPUs for model serving in OpenShift AI

Your first GPU algorithm: Scan/prefix sum

GPU benchmarking and how to choose a GPU framework

How to use AMD GPUs for model serving in OpenShift AI

Recent Posts

How I refactored a legacy Node.js test suite with Claude (and saved 3 days of work)

Install Red Hat Data Grid operator in a disconnected environment

Build resilient guardrails for OpenClaw AI agents on Kubernetes

Enable Firewall-as-a-Service in OpenStack Services on OpenShift

Platforms

Build

Quicklinks

Communicate

RED HAT DEVELOPER

Build here. Go anywhere.

We serve the builders. The problem solvers who create careers with code.

Join us if you’re a developer, software engineer, web designer, front-end designer, UX designer, computer scientist, architect, tester, product manager, project manager or team lead.

Sign me up →

About Red Hat

Jobs

Events

Locations

Contact Red Hat

Red Hat Blog

Inclusion at Red Hat

Cool Stuff Store

Red Hat Summit

© 2026 Red Hat

Privacy statement

Terms of use

All policies and guidelines

Digital accessibility

Cookie Preferences and Opt-Out Rights